Swedish Noun Phrase Tutor Agent

Architecture and Implementation Report

GitHub: `https://github.com/elintove/swedish-np-tutor` Video: `https://uppsalauniversitet-my.sharepoint.com/:v:/g/personal/elin_hoijer_6385_student_uu_se/IQB1hQq-84nUQqYemZwFOygUASQELT5ORrNuSy9F5W21iYI?e=QPHnkc`

Abstract

This report outlines the development and implementation of an intelligent tutoring agent designed to assist users in learning Swedish noun phrases (NPs). By integrating a Large Language Model (LLM) backend with a robust retrieval-augmented framework, the agent provides adaptive, context-aware grammar instructions. Key features include dynamic level placement, varied interaction modes, confusion-aware hints, and a BM25-based lexical retrieval system for answering grammar questions. Additionally, the system incorporates memory compaction to track long-term learner progress and adjust tutoring strategies dynamically.

1 Introduction

Learning the nuances of Swedish noun phrases, such as gender (en/ett), definiteness suffixes, and adjective agreement, can be a challenge for language learners. To address this, a Swedish Noun Phrase Tutor Agent was developed, in the shape of an interactive conversational system designed to guide learners through targeted exercises, adaptive feedback, and open-ended grammar questions. The project extends a foundational script into a fully functional, autonomous agent powered by a Large Language Model. All generated explanations are strictly presented in English to ensure clarity for novice learners. This report details the architectural enhancements, core features, and IR components integrated to make the system a robust and adaptable educational tool.

2 System Architecture

2.1 LLM Integration and Fallbacks

The core intelligence of the agent is driven by a real LLM client (`src/llm_client.py`) utilizing the **openai/gpt-oss-120b** model. It relies on a local environment loader (`src/env.py`) to securely manage API keys without requiring heavy external dependencies. To ensure the system remains operational under all conditions, a heuristic fallback mode is implemented. If the API key is missing or the LLM service is unavailable, the agent degrades to a rule-based system, relying on pre-defined grammar heuristics rather than generative text. The entry point of the application, `SweNPAgent.py`, orchestrates this initialization, setting up components like the `ErrorDetector` and `TutorGenerator` to utilize the LLM for producing structured JSON diagnostics and fluent tutoring explanations.

2.2 Information Retrieval and Knowledge Base

A vital requirement for the agent is its ability to reliably find and serve context to answer user questions and provide robust examples. I expanded the internal knowledge base (`grammar_db/`) to include comprehensive rules (`rules.json`), diverse examples (`examples.json`), and contrastive pairs (`minimal_pairs.json`). The agent covers four key grammatical concepts: definiteness mismatch, double definiteness, article omission, and adjective agreement.

To retrieve this information efficiently, a lightweight BM25-style lexical retriever was built within the retrieval module (`src/retrieval_module.py`). The retriever indexes rules, examples, and minimal pairs. When a user asks a question or triggers an error, the module ranks the available items and retrieves the most relevant context. To ensure varied and engaging sessions, the retrieval is non-deterministic, randomly selecting among the top matches to prevent repetitive examples. This layer ensures that the generated explanations are grounded in established linguistic rules.

2.3 Memory and State Management

I wanted the agent to be able retain context over time. Therefore, both short-term tracking and long-term memory compaction were implemented. Raw interactions, including placement results, exercises, free training inputs, and exam scores, are continuously appended to an event log (`data/session_log.jsonl`).

To prevent context fragility and enable cross-session continuity, a memory compactor (`src/memory_compactor.py`) periodically summarizes these events. Every ten logged interactions, or upon exiting the application, the compactor generates a concise learner profile (`data/learner_profile.md`). This profile captures the user's current level, top recurring errors (e.g., struggling with *ett* nouns), and recommendations for future focus. By condensing a noisy chat

history into a stable, easily retrievable profile, the agent maintains a reliable "working memory" that dynamically shapes the difficulty of future exercises.

3 Interaction Modes and Core Functionalities

The agent features several distinct interaction modes, each catering to a different phase of the user's learning process.

3.1 Dynamic Placement and Level Tracking

Upon starting, the agent greets the user and immediately initiates a translation-based placement test. It presents three English noun phrases for translation into Swedish. Based on the accuracy of the translations, it assigns the user to one of three proficiency levels: Advanced (0 errors), Intermediate (1 error), or Beginner (2–3 errors). Following the placement, the agent formulates a study plan tailored to the user's evaluated level before transitioning into the primary learning loop.

3.2 Learning and Free Training Modes

- **Learn Mode (`/learn`):** The default guided mode. The agent continuously provides level-appropriate English noun phrases for translation. The LLM evaluates the Swedish input. If incorrect, the agent provides an English explanation of the error and triggers a follow-up practice exercise. If correct, it confirms and advances. After every ten correct responses, the agent suggests the user attempt the exam mode to level up.
- **Free Training Mode (`/free`):** A flexible practice space where the user can submit any Swedish noun phrase. The agent evaluates the phrase, provides feedback, and generates a targeted follow-up exercise based on the specific input provided.

3.3 Question and Answer Mode (`/question`)

To fulfill the requirement of context-aware task performance, a dedicated Q&A mode allows learners to ask specific grammar questions (e.g., "Why is *en bilen* wrong?"). The system routes the question to the closest grammar topic using the BM25 retrieval module. It then synthesizes an answer using the LLM grounded in the retrieved lexical hits. For demonstration and transparency, the top retrieved hits and their BM25 scores are logged, explicitly showing the evidence the agent uses to formulate its response.

3.4 Exam Mode (`/exam`)

The exam mode strips away all hints and explanations. It presents a rigorous 5-item translation quiz where the agent responds strictly with "Correct" or "Incorrect." Passing the exam promotes the user's proficiency level. Achieving a perfect score at the Advanced level triggers a graduation sequence, celebrating the user as a "master of Swedish noun phrases."

4 Adaptive Tutoring Mechanisms

To emulate a human tutor, the agent implements "confusion-aware adaptive hints." By utilizing the state manager to track repeated errors per category, the tutor adjusts its verbosity dynamically:

- **First occurrence:** Provides a comprehensive explanation along with a contrastive minimal pair.
- **Second occurrence:** Offers a shorter, more direct hint.
- **Third and subsequent occurrences:** Reduces the feedback to a minimal cue (e.g., "Check noun gender (en/ett)").

This approach ensures the learner receives sufficient initial support but is pushed towards independent recall as they repeatedly encounter similar grammatical structures.

5 AI Tools Reflection

Initially, this project felt very overwhelming and I felt clueless to how to start it. I found the short video-lectures on the Studium Lab4 page helpful to start understanding the different pieces of AI Agents. Then, brainstorming ideas with Codex helped me understand what could realistically be done (which was a lot!). Codex helped me create different module scripts based on my ideas of how I wanted the final agent to function, which I reviewed, gave feedback to and adjusted until they fit my liking and the project frames. Codex also helped me tie everything together into a working main script.

I, unfortunately, would not have been able to implement nearly all of this myself, without the use of AI tools. However, being allowed to use these tools made the task manageable, and along the way I learned the architecture behind AI Agents, and how topics I, until now, had only heard about in lectures can be put into practical use.